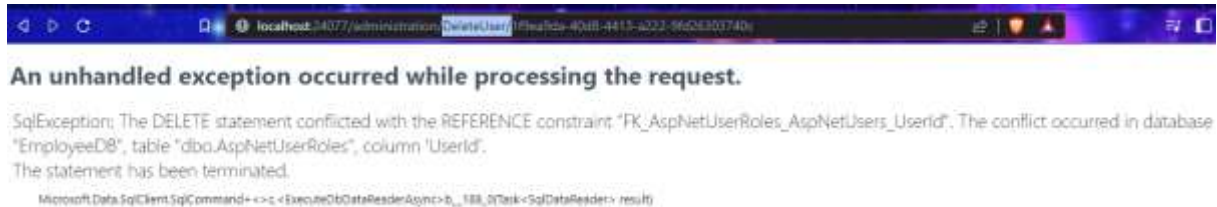
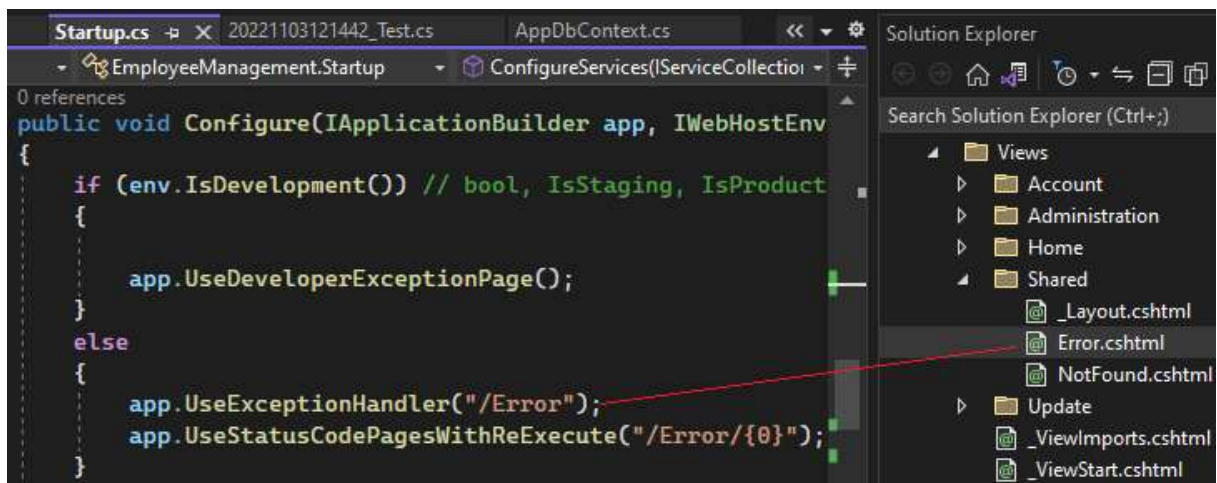


90 - Custom error page

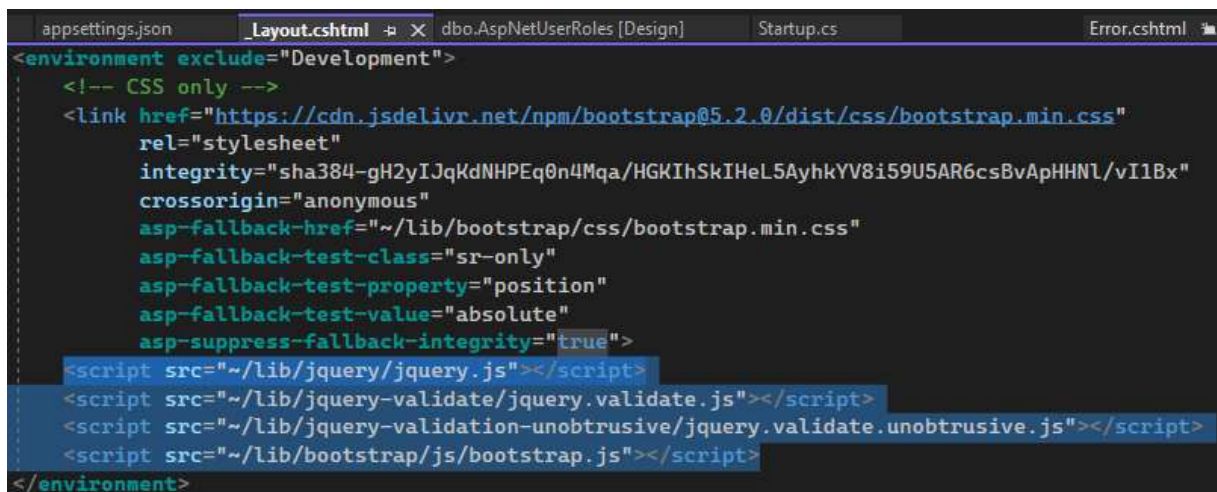
Това е продължение от предишния урок, при който при опит да изтрием Role, в който има потребители (не е празна), като защита на самата БД, On delete: No action, връща error page, която в DevelopmentEnvironment е ок, но за потребителите не, защото за обикновения потребител няма никакъв смисъл, а и е рисково за сигурността.

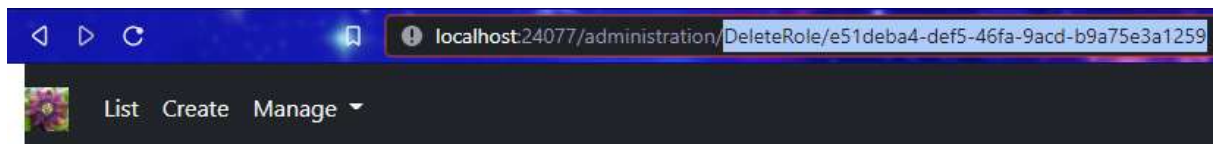


В Production искаме custom error page, която имаме налична, но трябва да минем в нейната среда, за да я видим.



Също, за да работи dropdown менюто, трябва да имаме възможност скриптовете да работят в тази среда:

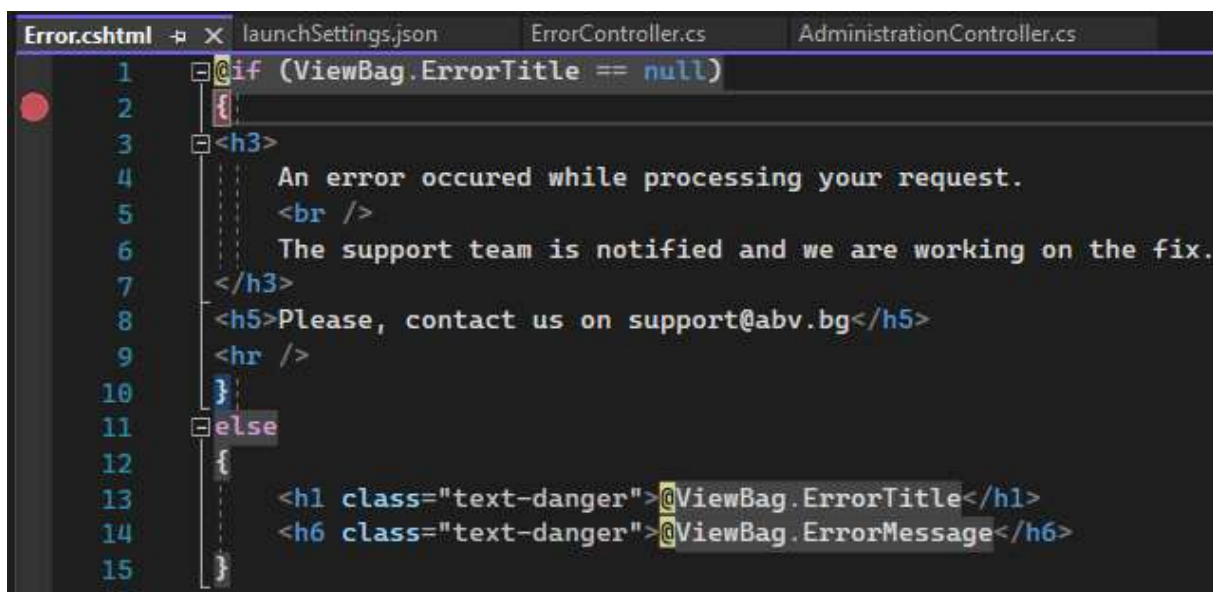




An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

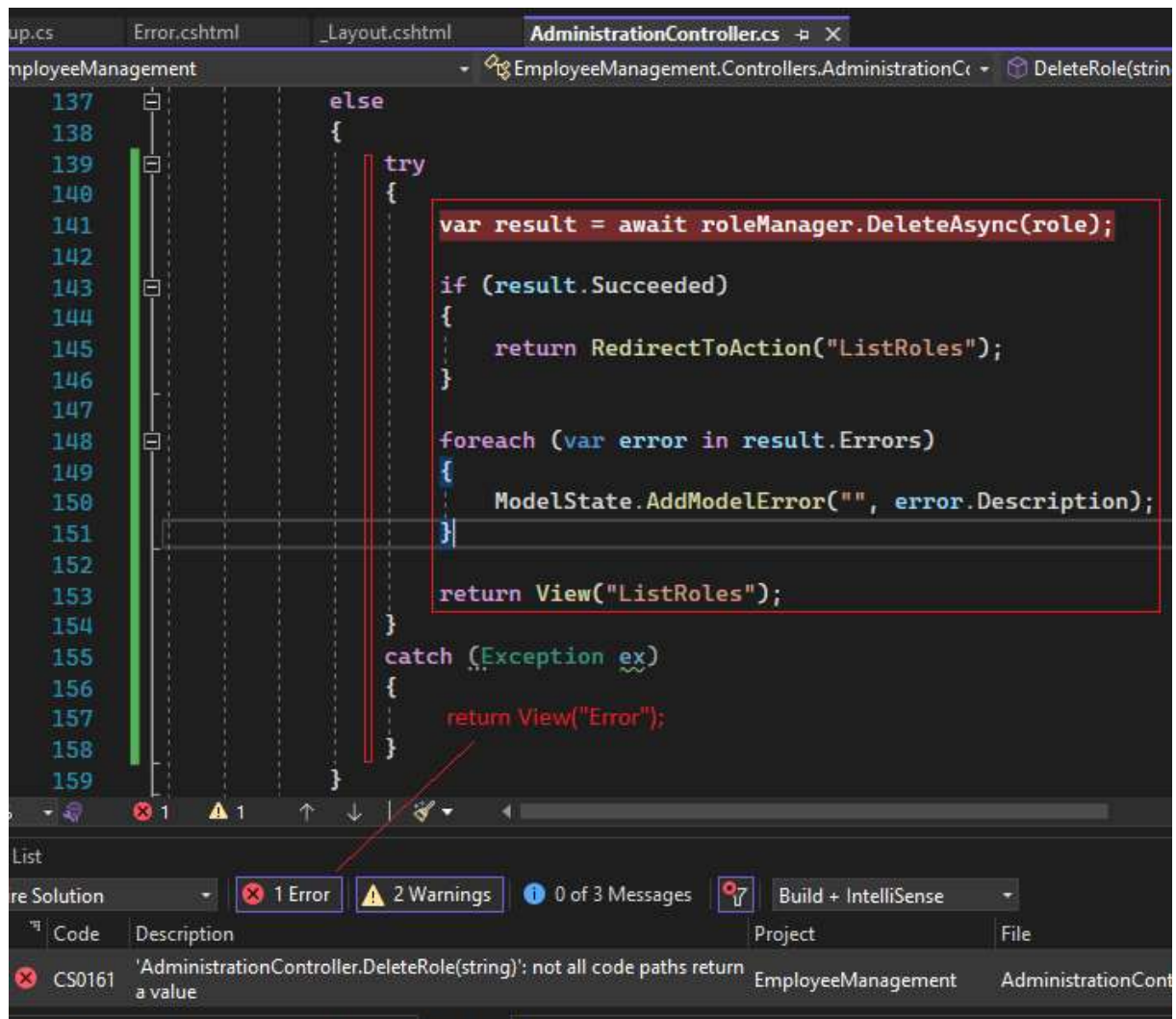
Това е прекалено генерална страница за грешка, а по-добре би било да имаме повече конкретна информация към потребителя:

Custom Error Page in ASP.NET Core



```
var result = await roleManager.DeleteAsync(role);
```

така че заграждаме следващите няколко реда в блок с `try{} + catch {}` за прихващане на изключение + breakpoint на този ред -> тестваме в debug mode.



Задейства се грешка, която се коригира с извикване на страницата Error.cshtml.

Сега остава въпросът да се прихванат стойности за

```
ViewBag.ErrorTitle = $"{role.Name} role is in use";
ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted as there are users
in this role. If you want to delete this role, please remove the users from the role and then try
to delete";
```

Където \$+ са знаците за нов ред

```

public async Task<IActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    else
    {
        try
        {
            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded)
            {
                foreach (var error in result.Errors)
                {
                    return View("ListRoles");
                }
            }
        }
        catch (Exception ex)
        {
            ViewBag.ErrorTitle = $"{role.Name} role is in use";
            ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted " +
                $"as there are users in this role. " +
                $"If you want to delete this role, please remove the users from the role " +
                $"and then try to delete";
            return View("Error");
        }
    }
}

```

The try-catch statement consists of a try block followed by one or more catch clauses, which specify handlers for different exceptions.

```

@if (ViewBag.ErrorTitle == null)
{
    <h3>
        An error occurred while processing your request.
        <br />
        The support team is notified and we are working on the fix.
    </h3>
    <h5>Please, contact us on support@abv.bg</h5>
    <hr />
}
else
{
    <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
    <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
}

```

BreakPoint: Поставяме на подозрителния ред в try:

```

var result = await roleManager.DeleteAsync(role);

```

Debug:

Опитваме да изтрием админ и потвърждаваме, при което командата ни изхвърля във VS:

```

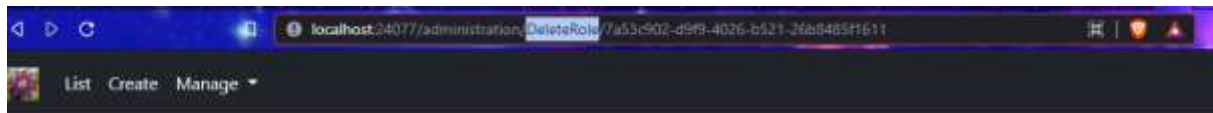
var result = await roleManager.DeleteAsync(role);

```

Поставяме нов BreakPoint, за да прескочи в catch:

```
282 | catch (Exception ex)
283 | {
284 |     logger.LogError($"Error deleting role : {ex}");
285 |
286 |     ViewBag.ErrorTitle = $"{role.Name} role is in use";
```

Continue Debugging



Admin role is in use

Admin role cannot be deleted as there are users in this role. If you want to delete this role, please remove the users from the role and then try to delete.

Променливата ex е със зелено подчертаване, понеже не се използва, като за да се коригира това, добавяме логър, който да проследи грешките в отделен файл. В реалните приложения, грешките се логват, за да може девелъпърът да ги коригира по-късно.

```
public AdministrationController(RoleManager<IdentityRole> roleManager,
                                UserManager<ApplicationUser> userManager,
                                ILogger<AdministrationController> logger)
{
    this.roleManager = roleManager;
    this.userManager = userManager;
    this.logger = logger;
}
```

```
private readonly UserManager<ApplicationUser> userManager;
private readonly ILogger<AdministrationController> logger;
```

Сега вече може да използваме logger за прихващане на изключението.

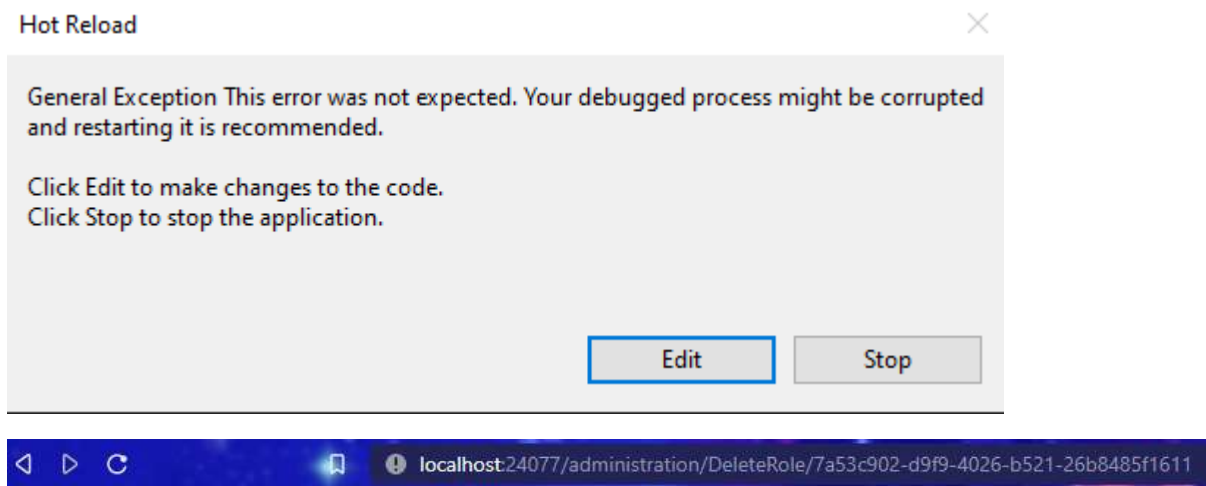
За демонстрация, поставяме ред

```
throw new Exception("Test exception");
```

непосредствено след try {, с BreakPoint на него -> Debug:

```
144 | try
145 | {
146 |     throw new Exception("Test exception");
147 | }
```

В този случай, Exception е general - Continue ->



В новите версии се появява това съобщение за редакция на кода, а в старите просто се зарежда старата страница със старото съобщение за грешка.

По-добрият вариант, вместо генералния `catch (Exception ex)` -> специфичния `catch (DbUpdateException ex)`, защото иначе всеки път ще прихваща всички видове грешки и връща червената страница. Искаме в общия случай черната, а при определени обстоятелства само червената. Направим ли го, вече `catch` ще пресее грешката, ще види, че не може да я обработи и ще я препрати към `ErrorController`, който ще я запише в `log` файла и препрати към `Error view`, без да присвои стойност на `ErrorTittle`, който ще остане `null`, при което `if == true` и ще се изведе черното съобщение. Всичко това е, за да връща черната страница в общия случай, а само при опит за изтриване на заета `role` – червената с информативното съобщение за корекция.

Нека сменим:

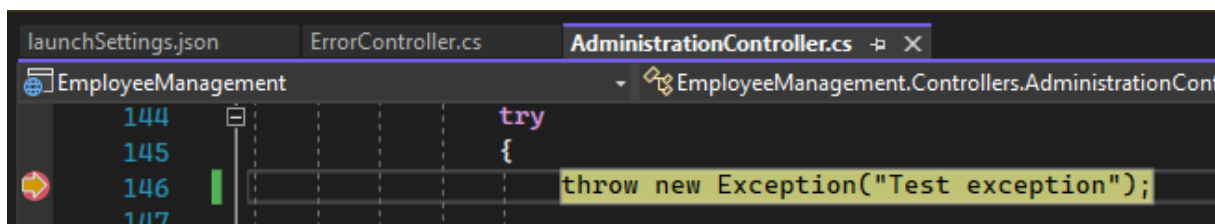
```
catch (DbUpdateException ex)
```

По този начин ще бъдат прихванати с `catch` само специфичните за БД грешки, а не всички `general`.

Стартираме по същият начин проекта в `Debug mode` и стартира страницата с желаното съобщение.

Правим 3-та демонстрация: `Debug`, после нов `BreakPoint` в `ErrorController`,

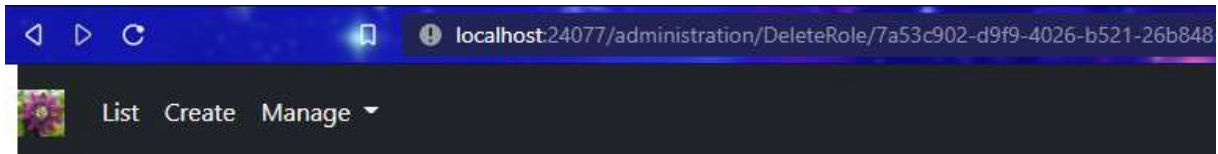
Потвърждаваме изтриване на Админ, за да видим какво става. Изхвърля ни към



В моята версия отново извежда съобщение за редакция.

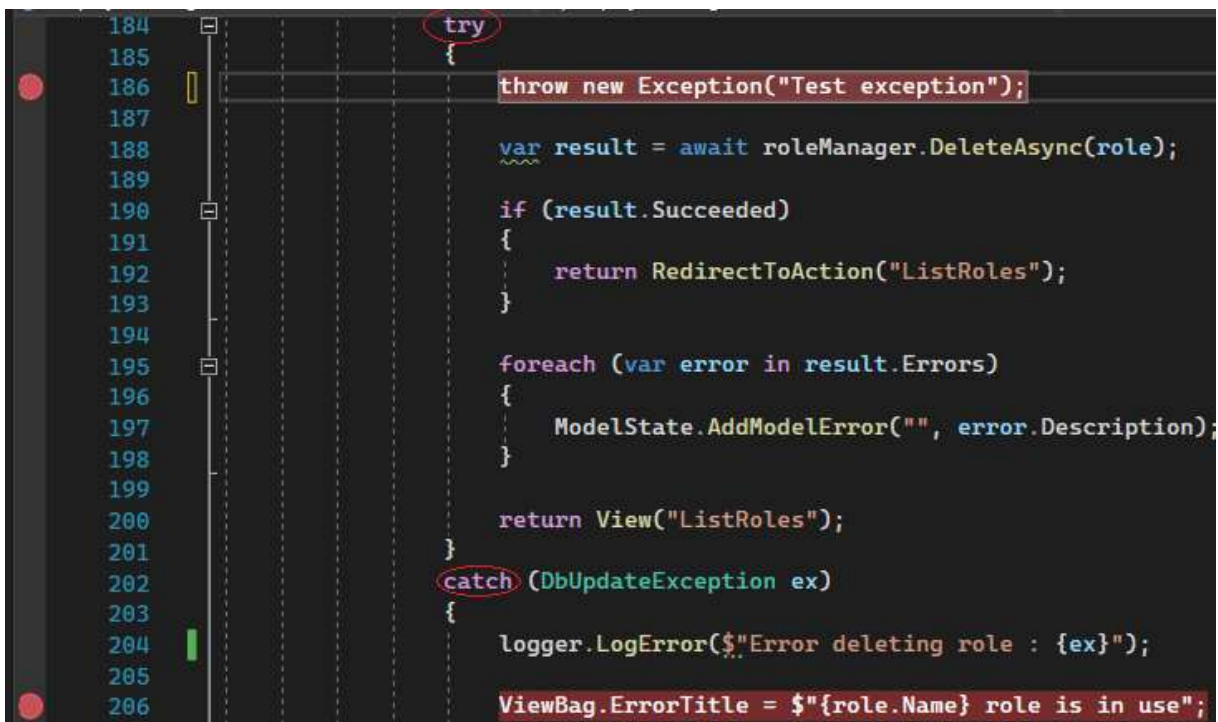
В този случай, нямаме подадена стойност за ViewBag.ErrorTitle – както се вижда при посочване, тя е null, така че се изпълнява кодът от if блока с обикновеното съобщение за грешка.

При активация на BreakPoint, изпълнението е с жълта стрелка. Поставяме нова BreakPoint -> Continue:



An error occurred while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

Последно оставяме няколко Breakpoint:



```
ErrorController.cs
0 references
public IActionResult Error()
{
    var exceptionDetails = HttpContext.Features.Get<IExceptionHandler>();

    logger.LogError($"The path {exceptionDetails.Path} " +
        $"threw an exception {exceptionDetails.Error}");

    ViewBag.ExceptionPath = exceptionDetails.Path;
    ViewBag.ExceptionMessage = exceptionDetails.Error.Message;
    ViewBag.Stacktrace = exceptionDetails.Error.StackTrace;

    return View("Error");
}
```

```
Error.cshtml
1 @if (ViewBag.ErrorTitle == null)
2 {
3     <h3>
4         An error occured while processing your request.
5         <br />
6         The support team is notified and we are working on the fix.
7     </h3>
8     <h5>Please, contact us on support@abv.bg</h5>
9     <hr />
10 }
11 else
12 {
13     <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
14     <h2 class="text-danger">@ViewBag.ExceptionMessage</h2>
15 }
```

Като всеки Continue ни прехвърля от един BreakPoint към следващ и накрая, понеже ErrorTitle == null, виждаме обикновения NotFound View, за general Errors.



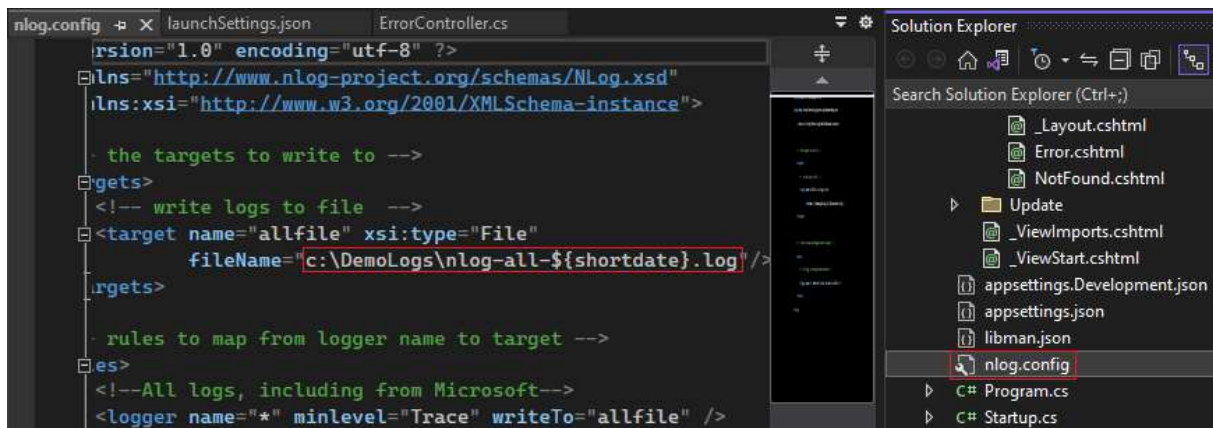
List Create Manage ▾

An error occured while processing your request.
The support team is notified and we are working on the fix.
Please, contact us on support@abv.bg

Закоментираме вече излишния ред

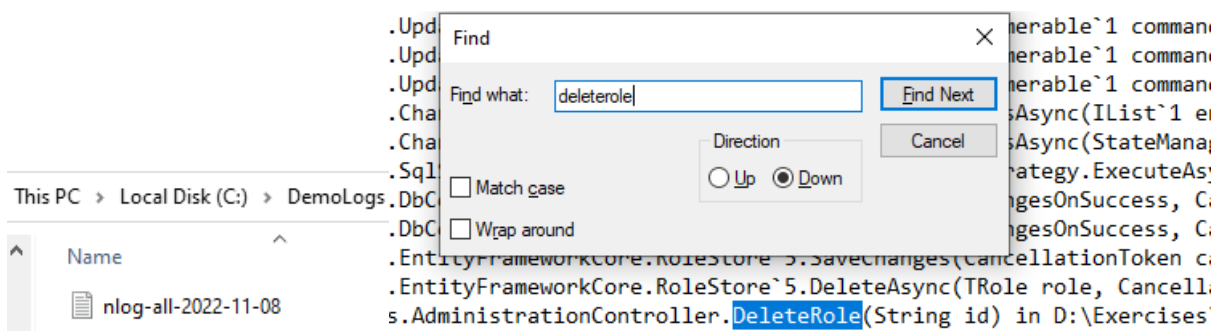
```
//throw new Exception("Test exception");
```

При тези обстоятелства, ако се опитаме в Debug mode да изтрием админ, се отваря персонализираната страница **Admin role cannot be deleted as there are users in this role** и се отчита в nlog.config, чието местоположение може да намерим:



Може в него да потърсим DeleteRole или съобщението, което сме изписали:

```
logger.LogError($"Error deleting role : {ex}");
```



[Microsoft.EntityFrameworkCore.DbUpdateException]: An error occurred while saving the entity changes.

```
[HttpPost]
public async Task<IActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null)
    {
        ViewBag.ErrorMessage = $"Role with Id = {id} cannot be found";
        return View("NotFound");
    }
    else
    {
        try
        {
            //throw new Exception("Test exception");

            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded)
            {
                return RedirectToAction("ListRoles");
            }

            foreach (var error in result.Errors)
            {
                ModelState.AddModelError("", error.Description);
            }

            return View("ListRoles");
        }
    }
}
```

```

        catch (DbUpdateException ex)
        {
            logger.LogError($"Error deleting role : {ex}");

            ViewBag.ErrorTitle = $"{role.Name} role is in use";
            ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted "
+
            $"as there are users in this role. " +
            $"If you want to delete this role, please remove the
users from the role " +
            $"and then try to delete";
            return View("Error");
        }
    }
}

```

Custom Error Page in ASP.NET Core



```

if (ViewBag.ErrorTitle == null)
{
    <h3>
        An occurred while processing your request.
        The support team is notified and we are working on the fix
    </h3>
    <h5>Please contact us on pragim@pragimtech.com</h5>
}
else
{
    <h1 class="text-danger">@ViewBag.ErrorTitle</h1>
    <h6 class="text-danger">@ViewBag.ErrorMessage</h6>
}

```

```
[HttpPost]
public async Task<IActionResult> DeleteRole(string id)
{
    var role = await roleManager.FindByIdAsync(id);

    if (role == null) ...
    else
    {
        try
        {
            var result = await roleManager.DeleteAsync(role);

            if (result.Succeeded) ...

            foreach (var error in result.Errors) ...

            return View("ListRoles");
        }
        catch (DbUpdateException ex)
        {
            logger.LogError($"Exception Occured : {ex}");
            ViewBag.ErrorTitle = $"{role.Name} role is in use";
            ViewBag.ErrorMessage = $"{role.Name} role cannot be deleted";
            return View("Error");
        }
    }
}
```

Controller Action